Side Activity 5: UPDATE & ALTER TABLE

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, we focus on two practical skills: **updating data** and **schema evolution**. You will learn:

- How to use `UPDATE` to modify existing rows in Oracle.
- How to update **one row vs many rows**, and how to update using **expressions**.
- A critical safety rule: **always check your WHERE clause**. Without `WHERE`, `UPDATE` modifies **all rows**.
- How to evolve table structure with `ALTER TABLE`: add/modify columns, defaults, constraints, renaming, and dropping.
- **ALTER conflicts**: new constraints conflicting with existing data (duplicates / NULLs), and how to diagnose and fix the data before retrying.

Part 0: Getting started

Please refer to the instructions in **Side Activity 3** to access your Oracle environment.
For a comprehensive list of additional SQL*Plus commands and syntax, please consult the supplementary file `notes_from_lecturer.pdf`.

Part 1: `UPDATE ... SET ... WHERE ...` (do in Oracle SQL*Plus)

Key Concept: What does UPDATE do?

`UPDATE` modifies **existing rows** in a table. It does not create new rows (that is `INSERT`) and it does not remove rows (that is `DELETE`).

Step 1: Create a simple table and insert some values

We will reuse this table later in Part 2 (ALTER TABLE). If already deleted, create it again.

```
CREATE TABLE orders (
  order_id   INTEGER PRIMARY KEY,
  status     VARCHAR2(20),
  qty        INTEGER
);

INSERT INTO orders VALUES (1, 'CREATED', 1);
INSERT INTO orders VALUES (2, 'CREATED', 3);
INSERT INTO orders VALUES (3, 'CANCELLED', 2);
INSERT INTO orders VALUES (4, NULL, 2);
```

Step 2: Update a single row (WHERE by primary key)

```
UPDATE orders
SET status = 'SHIPPED'
WHERE order_id = 1;
```

Step 3: Update many rows (WHERE by condition)

```
UPDATE orders
SET status = 'CREATED'
WHERE status IS NULL;
```

Step 4: Update using an expression

```
UPDATE orders
SET qty = qty + 1
WHERE status = 'CREATED';
```

Step 5: Verify your updates

```
SELECT * FROM orders ORDER BY order_id;
```

Safety rule: always check WHERE

Never run `UPDATE` without `WHERE` unless you intentionally want to update **every row** in the table.

Part 2: More on `ALTER TABLE` (Schema Evolution)

Key Concept: Why is ALTER more risky?

`ALTER TABLE` changes the **schema**, not just a few rows. A wrong schema change can break many queries and application code. A safe workflow is: **prepare/clean data** → **apply schema/constraints** → **verify**.

Step 1: Add a column (ADD)

```
ALTER TABLE orders ADD (
  carrier VARCHAR2(30)
);
```

Step 2: Set a default (DEFAULT)

```
ALTER TABLE orders MODIFY (
  status DEFAULT 'CREATED'
);
```

Step 3: Make a column NOT NULL (a typical conflict)

Conflict reason: you want `NOT NULL`, but existing rows contain NULL.

Try it first (it will fail):

```
ALTER TABLE orders MODIFY (status NOT NULL);
```

Expected error message

If there are NULL values in `orders.status`, Oracle will reject the change. A common error is: `ORA-02296: cannot enable (...) - null values found`.

```
-- Check whether NULL exists
SELECT COUNT(*) AS null_cnt
FROM orders
WHERE status IS NULL;

-- If NULL exists, fix data first
UPDATE orders SET status = 'CREATED' WHERE status IS NULL;

-- Then add NOT NULL
ALTER TABLE orders MODIFY (status NOT NULL);
```

Step 4: Add constraints and name them

Use readable constraint names to simplify troubleshooting and dropping later:

```
ALTER TABLE orders
ADD CONSTRAINT orders_status_ck
CHECK (status IN ('CREATED', 'SHIPPED', 'DELIVERED', 'CANCELLED'));
```

Step 5 (important): conflict example — UNIQUE fails due to duplicates

This is one of the most common ALTER conflicts: adding `UNIQUE(email)` fails because the table already contains duplicate emails.

```
CREATE TABLE users_raw (
  user_id INTEGER PRIMARY KEY,
  email   VARCHAR2(100),
  name    VARCHAR2(50)
);

INSERT INTO users_raw VALUES (1, 'alice@demo.com', 'Alice');
INSERT INTO users_raw VALUES (2, 'alice@demo.com', 'Alice-dup');
INSERT INTO users_raw VALUES (3, NULL,             'NoEmail');
```

Try to add UNIQUE (will conflict):

```
ALTER TABLE users_raw
ADD CONSTRAINT users_email_uk UNIQUE (email);
```

What will you see?

A common Oracle error is: `ORA-02299: cannot validate (..USERS_EMAIL_UK) - duplicate keys found.`

Find duplicates:

```
SELECT email, COUNT(*) AS cnt
FROM users_raw
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Fix A (keep one row, delete the rest):

```
DELETE FROM users_raw
WHERE email IS NOT NULL
  AND user_id NOT IN (
    SELECT MIN(user_id)
    FROM users_raw
    WHERE email IS NOT NULL
    GROUP BY email
  );

  -- Add UNIQUE again (should succeed)
ALTER TABLE users_raw
ADD CONSTRAINT users_email_uk UNIQUE (email);
```

Fix B (business requires keeping duplicates): Do not force a UNIQUE constraint. You may need a composite uniqueness rule (e.g., **(email, name)**), or introduce a true business key (e.g., `username`) and make that UNIQUE instead.

Step 6: Drop constraints

```
ALTER TABLE orders DROP CONSTRAINT orders_status_ck;
```

Step 7: Rename column/table

```
-- Rename a column
ALTER TABLE orders RENAME COLUMN qty TO quantity;

-- (Optional) Rename a table
-- RENAME orders TO orders_v2;
```

Step 8: Drop a column

```
ALTER TABLE orders DROP COLUMN carrier;
```

Part 3: Self Challenge Exercises (not graded)

Task 1: UPDATE (`UPDATE ... SET ... WHERE ...`)

Goal: practice safe and correct `UPDATE` statements in SQL*Plus.

1. Create a small database with multiple related tables:
   - `customers_t(customer_id PK, email, name)`
   - `products_t(product_id PK, product_name, unit_price, stock_qty)`
   - `orders_t(order_id PK, customer_id FK, status, total_amount, created_at)`
   - `order_items_t(order_id FK, product_id FK, qty, unit_price)` with a composite primary key `(order_id, product_id)`
2. Insert baseline data. Your data must include:
   - At least 1 order with `status IS NULL`
   - At least 1 product with `stock_qty = 0`
   - At least 1 product with `stock_qty < 0`
   - At least 2 orders with multiple `order_items_t` rows
3. Before every UPDATE you write, first provide a **preview SELECT** that shows which rows will be affected.
   - Please include one example of an affected tuple as an SQL comment.
4. Write the following UPDATE statements (each followed by a verification SELECT):
   - Update exactly one order row by primary key (e.g., set `status='SHIPPED'` for a specific `order_id`).
   - Cancel stale orders: set `status='CANCELLED'` where `status='CREATED'` and `created_at < SYSDATE - 5`.
   - Fix invalid stock: set `stock_qty=0` where `stock_qty < 0`.
   - Restock out-of-stock products: set `stock_qty=10` where `stock_qty = 0`.
   - Apply a product price change: increase `unit_price` by 10% where `stock_qty <= 3`.
   - Fill missing `orders_t.total_amount`: set `total_amount=0` where `total_amount IS NULL`.

Task 2: ALTER TABLE (with conflicts and resolutions)

Goal: start from "dirty data", then add constraints via ALTER and resolve the resulting conflicts correctly.

1. From the prev. created table `customers_t(customer_id PK, email, name)`, give your SQL command(s) that insert "dirty" data, they must satisfy the following:
   - At least 1 row with `email IS NULL`;
   - At least 1 set of **duplicate emails** (two or more rows with the same email).
   - At least 1 row with an invalid email format (e.g., missing `@`).
2. **Conflict 1 (NOT NULL)**: Let's now attempt to modify the column constraints such that `email` is now NOT NULL (it should fail or be invalid before cleaning). Provide the following SQL commands:

- A query to find NULL emails;
- An UPDATE to fix them (use reasonable values);
- A successful `ALTER TABLE ... MODIFY(email NOT NULL)` and verification.

3. **Conflict 2 (UNIQUE)**: Let's now attempt to modify the column email such that is unique. i.e., add `UNIQUE(email)` (duplicates cause conflicts). Provide the following SQL commands:
   - A query to finding any duplicates (Hint: you will need `GROUP BY ... HAVING ...` and `COUNT()`);
   - A fix (delete extra rows or update emails to make them unique);
   - A successful UNIQUE constraint add and verification.
4. **Conflict 3 (CHECK)**: Let's now add a CHECK constraint that enforces a basic email rule (for example, `email LIKE '%%'`). If will fail without prior cleaning, provide the following SQL commands:
   - A query that finds rows violating the rule before any modification is down;
   - A fix (UPDATE) and then add the CHECK constraint successfully.
5. Additional schema changes (do at least 3):
   - Add a column with a DEFAULT (e.g., `created_at DATE DEFAULT SYSDATE`).
   - Rename a column (e.g., `name -> full_name`).
   - Drop a constraint by name (choose one you added).
6. Explore modifying Foreign key rules (Oracle **ON DELETE** actions). Provide the following SQL commands:
   - Give the commands that create two tables:
     - `dept_t(dept_id PK, dept_name)`
     - `emp_t(emp_id PK, emp_name, dept_id)` where `dept_id` is allowed to be NULL
     - Note: do not include any foreign key rules here yet.
   - With `ALTER TABLE`, give your SQL command(s) to define a foreign key with:
     - `ON DELETE CASCADE`
   - With `ALTER TABLE` again, change the foreign key so that it becomes:
     - `ON DELETE SET NULL`
   - **Important (Oracle):** `ON UPDATE CASCADE` is **not supported**. If you need to change a referenced key value, you must handle the child table updates manually (or avoid updating primary keys in your schema design).
   - (Hint: you will need `DROP CONSTRAINT` and `ADD CONSTRAINT`.)

Task 3: Clean up

At the end of your file, drop all tables you created in Task 1 and Task 2 (e.g., `DROP TABLE order_items_t;`, `DROP TABLE orders_t;`, `DROP TABLE products_t;`, `DROP TABLE customers_t;`). If a table does not exist, you may skip it.

End of Side Activity 5.